STUDENT INTERNSHIP & PLACEMENT TRACKER WITH AI-BASED SUITABILITY PREDICTION

A PROJECT REPORT

Submitted by

SUJIT S
(240701545)

in partial fulfilment of the End Semester Examination for the course

CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF ENGINEERING
COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE
(An Autonomous Institution, Affiliated to Anna University, Chennai)
Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled “Student Internship & Placement Tracker with AI-based Suitability Prediction” is the bonafide work of **SUJIT S (240701545)**, who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr. J. Manoranjini
HEAD OF THE DEPARTMENT
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Mr. B. Bhuvaneswaran
COURSE FACULTY
Assistant Professor (SG)
Department of Computer Science and Engineering
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, **SUJIT S (240701545)**, hereby declare that the project report titled “Student Internship & Placement Tracker with AI-based Suitability Prediction”, submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, page, Deluge scripts, workflows, integrations, Custom API and AI prediction model — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia keyword extraction or the Zia AI Modeler), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai

Date: _____

SIGNATURE OF THE STUDENT

SUJIT S

240701545

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman Thiru S. Meganathan, Chairperson Dr. (Mrs.) Thangam Meganathan and Vice-Chairman Mr. Abhay Shankar Meganathan for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr. S. N. Murugesan, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. J. Manoranjini, Head of the Department, Department of Computer Science and Engineering, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Mr. B. Bhuvaneswaran, Assistant Professor (SG), for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

SUJIT S

ABSTRACT

College placement cells manage student profiles, company drives, applications, interview rounds and offers with spreadsheets, online forms and e-mail. Because the data is copied between several files, students apply to closed or expired drives or below the CGPA cut-off, duplicate applications slip through, interview schedules and status changes are not communicated consistently, offers are retyped into separate sheets, and management has no live view of placement statistics.

The **Student Internship & Placement Tracker** is a Zoho Creator application that manages the complete placement cycle: student registration, companies, opportunities, applications, interviews, placement results and an AI-assisted suitability check. It has seven linked forms with six lookups, nine reports (list, Kanban and chart), a KPI dashboard page, eleven form workflows, one scheduled workflow and four Deluge functions. A Deluge validation blocks applications to non-open or expired opportunities, from students below the minimum CGPA and duplicates, while workflows e-mail students and the officer, update application statuses, create a Zoho Calendar event for every interview and export every placement result to Zoho Sheet without duplicates, both through OAuth connections. A secured **Custom API**, Placement Summary (GET, OAuth2, admin only), publishes department-wise placement statistics as JSON.

For AI decision support, Zia keyword extraction reads each job description and a transparent rule turns the matched skills and CGPA into a suitability score and recommendation, and a **Zia prediction model**, Suitability_Prediction, trained on 300 synthetic records, predicts whether an application is likely to be Selected from department, year, CGPA, required CGPA and skill-match percentage. The AI output is advisory and never changes an application's status.

The application was tested with 17 test cases, of which 14 passed, one was verified by code review and two could not be run; every integration was verified in the receiving service. A sample student scored 70.00 (Strong Match), and the prediction model agreed with its generating rule in 7 of 8 live spot tests.

Keywords: *Low-Code Development; Zoho Creator; Deluge Scripting; Custom API; Zia AI; Placement Management*

PROJECT AT A GLANCE

Attribute	Details
Project Title	Student Internship & Placement Tracker with AI-based Suitability Prediction
Application Domain	Education – college placement and internship management
Student Name & Register No.	SUJIT S – 240701545
Zoho Creator Edition	Zoho Creator, India data centre (creatorapp.zoho.in)
Application Name (Link Name)	Student Internship & Placement Tracker app (student-internship-placement-tracker-app)
Published / Live URL	https://creatorapp.zoho.in/sujit.s.2024.cse_rajalaks896/student-internship-placement-tracker-app
Demo Credentials (per role)	Administrator: owner Zoho account (signed in during the demonstration); no separate role accounts – see Section 5.8
Number of Forms / Subforms	7 / 0
Number of Reports (by type)	List: 5 Kanban: 3 Chart: 1 Calendar/Pivot: 0
Number of Pages / Dashboards	1 (Placement_Dashboard)
Number of Workflows	Form: 11 Schedule: 1 Approval: 0 Blueprint: 0 Batch: 0
Deluge Functions / Custom Actions	4 functions / 15 Deluge scripts in total; no custom report actions
Integrations	Zoho Mail (sendmail), Zoho Calendar and Zoho Sheet (OAuth connections); Custom API “Placement Summary”; Zia keyword extraction and Zia prediction model
Roles & Profiles Defined	Administrator (implemented); Read, Write and Student permission profiles
Environments Used	Development (Stage and Production not used)
Date of Production Deployment	Live in the Development environment since 10 November 2026

TABLE OF CONTENTS

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES viii
LIST OF FIGURES ix
LIST OF ABBREVIATIONS x
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 2
1.6 Organisation of the Report 2
LITERATURE SURVEY AND SYSTEM STUDY 3
2.1 Study of the Existing System 3
2.2 Limitations of the Existing System 3
2.3 Low-Code versus Traditional Development 3
2.4 Literature Survey 4
2.5 Emerging Trends in Low-Code Development 4
2.6 Proposed System 4
REQUIREMENT ANALYSIS 5
3.1 Stakeholders and User Roles 5
3.2 Functional Requirements 5
3.3 Non-Functional Requirements 6
3.4 Hardware and Software Requirements 6
3.5 Use Case Model 7
SYSTEM DESIGN 8
4.1 System Architecture 8
4.2 Data Model 8
4.3 Form Design 9
4.4 Process and Workflow Design 11
4.5 User Interface and Navigation Design 11
4.6 Access Control Design 12
IMPLEMENTATION 13
5.1 Application Setup 13
5.2 Forms and Data Capture 13
5.3 Reports 14
5.4 Pages and Dashboards 14
5.5 Deluge Scripting 15
5.6 Workflows and Automation 16

5.7 Integrations and Custom API 17
5.8 User Management and Role-Based Access 20
5.9 AI and Analytics Features 20
5.10 Pro-Code Extension 23
5.11 Mobile and Portal Access 23
TESTING AND DEBUGGING 24
6.1 Testing Strategy 24
6.2 Test Cases 24
6.3 Debugging Deluge Scripts 25
6.4 Defect Log 25
6.5 User Acceptance Testing 26
DEPLOYMENT, SECURITY AND GOVERNANCE 27
7.1 Environments and Deployment Strategy 27
7.2 Performance Optimisation 27
7.3 Security Features 28
7.4 Governance and Administration 28
RESULTS AND DISCUSSION 29
8.1 Application Output 29
8.2 Achievement of Objectives 30
8.3 Comparison with the Existing System 30
8.4 Limitations of the Low-Code Approach 30
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 30
CONCLUSION AND FUTURE ENHANCEMENTS 31
9.1 Conclusion 31
9.2 Future Enhancements 31
9.3 Learning Outcomes and Career Relevance 31
REFERENCES 32
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 33
APPENDIX B – COURSE OUTCOME MAPPING 34
APPENDIX C – COMPLETE DELUGE SOURCE CODE 35
APPENDIX D – USER MANUAL 41
D.1 Accessing the Application 41
D.2 Instructions for the Placement Officer 41
D.3 Instructions for Students 41
APPENDIX E – EVALUATION RUBRIC 42
APPENDIX F – VIVA-VOCE RECORD SHEET 44

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application 3
Table 2.2: Summary of Literature Survey 4
Table 3.1: Stakeholders and User Roles 5
Table 3.2: Functional Requirements 5
Table 3.3: Non-Functional Requirements 6
Table 3.4: Hardware and Software Requirements 6
Table 3.5: Use Case Description – Submit Application 7
Table 4.1: Entities and Relationships 8
Table 4.2: Field Specification – Students Form 9
Table 4.3: Field Specification – Opportunities Form 10
Table 4.4: Field Specification – Applications Form 10
Table 4.5: Summary of All Forms 10
Table 4.6: Role–Permission Matrix 12
Table 5.1: Reports Implemented 14
Table 5.2: Deluge Scripts Implemented 15
Table 5.3: Workflows Implemented 16
Table 5.4: Integrations Implemented 17
Table 5.5: Custom API Configuration 19
Table 5.6: Zia Prediction Model Details 22
Table 5.7: Spot Test of the Prediction Model (8 Cases Entered in the Live Form) 23
Table 6.1: Test Cases 24
Table 6.2: Defect Log 25
Table 6.3: User Acceptance Feedback 26
Table 7.1: Deployment Checklist 27
Table 7.2: Performance Optimisation Measures (timings not measured) 27
Table 7.3: Security Controls Implemented 28
Table 8.1: Mapping of Objectives to Outcomes 30
Table 8.2: Existing System versus Proposed Application 30
Table A.1: Feature Coverage Checklist 33
Table B.1: Mapping of Course Outcomes to Report Chapters 34
Table E.1: Evaluation Rubric for the Individual Project 42
Table E.2: Performance Level Descriptors 43
Table F.1: Viva-Voce Questions and Responses 44
Table F.2: Marks Summary 44

LIST OF FIGURES

Figure 3.1: Use Case Diagram of the Application 7
Figure 4.1: System Architecture of the Application 8
Figure 4.2: Entity–Relationship Diagram 9
Figure 4.3: Workflow / Process Flow of an Application 11
Figure 4.4: Navigation Structure of the Application 12
Figure 5.1: Application Schema in the Schema Builder 13
Figure 5.2: Students Form in Live Mode 13
Figure 5.3: Applications_Report (List Report) 14
Figure 5.4: Companies_Report (Kanban by Industry) 14
Figure 5.5: Placement_Dashboard: KPI Cards, Quick Links and Chart 15
Figure 5.6: Deluge Functions of the Application 16
Figure 5.7: Form Workflows Configured on the Applications Form 17
Figure 5.8: Connection Setup – Zoho Calendar (OAuth) 18
Figure 5.9: Custom API Builder – Placement Summary 19
Figure 5.10: Execution Output of getPlacementSummary 20
Figure 5.11: Permission Profiles: Read, Student and Write 20
Figure 5.12: Suitability Result: 70.00, Strong Match, with Matched and Missing Skills 21
Figure 5.13: Zia Prediction Model Details in the AI Modeler 22
Figure 5.14: Suitability_Predictions Form with Auto-Filled Inputs and ML Outcome 22
Figure 6.1: Validation Alert for CGPA below the Minimum (TC-03) 25
Figure 7.1: Environments – Development, Stage and Production 27
Figure 8.1: Scenario Step 2 – Interview Event Created in Zoho Calendar 29
Figure 8.2: Scenario Step 3 – Placement Results Exported to Zoho Sheet 29
Figure 8.3: Scenario Step 4 – Applications by Status on the Dashboard 29

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
CGPA	Cumulative Grade Point Average
CSE	Computer Science and Engineering
DS	Deluge Script
ER	Entity–Relationship
FK / PK	Foreign Key / Primary Key
FR / NFR	Functional / Non-Functional Requirement
HTML	HyperText Markup Language
IST	Indian Standard Time
IT	Information Technology
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
LPA	Lakhs Per Annum
ML	Machine Learning
NLP	Natural Language Processing
OAuth	Open Authorization
RBAC	Role-Based Access Control
REST	Representational State Transfer
TC	Test Case
UAT	User Acceptance Testing
Zia	Zoho's AI assistant

CHAPTER 1
INTRODUCTION

1.1 Background

Every engineering college runs a placement cell that connects students with companies offering internships and full-time jobs. In a typical semester the cell registers hundreds of student profiles, receives job descriptions from recruiters, collects applications, arranges several interview rounds and records offers and joining status. Most cells still do this with shared spreadsheets, Google Forms, e-mail threads and WhatsApp groups, so the same data is copied many times and nobody has a single, current view of who applied where and what happened next.

The **Student Internship & Placement Tracker** is a Zoho Creator application that brings this whole cycle into one place: student registration, company and opportunity management, application screening, interview scheduling, placement results, an AI-assisted suitability check and a live placement dashboard.

1.2 Problem Statement

Placement data is spread across spreadsheets and mailboxes. As a result:

- students apply to closed or expired drives, or apply twice to the same drive, and ineligible applications (CGPA below the cut-off) are noticed only at shortlisting;
- students are not told consistently when their status changes or when an interview is fixed;
- interview slots are not on anybody's calendar and results are copied by hand into a separate offer sheet;
- officers judge a student's fit for a role by reading each job description and résumé manually;
- management has no live count of applications, interviews and placements by department.

1.3 Objectives

- To design a normalised data model of seven linked forms covering students, companies, opportunities, applications, interviews, placement results and suitability predictions.
- To enforce eligibility rules (open drive, deadline, minimum CGPA, no duplicates) with Deluge validation before an application is saved.

- To automate status updates and e-mail notifications with form and scheduled workflows.
- To integrate Zoho Calendar (interview events) and Zoho Sheet (placement export) through OAuth connections.
- To expose read-only placement statistics through a secured Custom API.
- To add AI decision support: Zia keyword extraction for skill matching and a Zia prediction model that predicts the likely outcome of an application.
- To provide list, Kanban and chart reports and a KPI dashboard page for the placement officer.

1.4 Scope of the Project

In scope: student, company and opportunity masters; validated applications; interview and result tracking; e-mail notifications; Zoho Calendar and Zoho Sheet integrations; one Custom API; Zia keyword extraction and a Zia prediction model; reports, a dashboard page and the Creator mobile app. **Out of scope:** a public recruiter portal, résumé parsing, online assessments, SMS/WhatsApp messages and automatic shortlisting — the AI output is advisory and never changes an application status by itself. **Users:** the placement officer (administrator) and students.

1.5 Justification for a Low-Code Approach

A placement cell needs a secure, hosted, multi-user application quickly, but does not have a software team. Low-code platforms shorten delivery by replacing hand-written plumbing with configuration [3], [4]. Zoho Creator supplies hosting, authentication, forms with lookups, several report types, workflows, OAuth connections, Custom APIs and Zia AI out of the box [5]; only the placement rules, integrations and AI scoring logic had to be written in Deluge [6].

1.6 Organisation of the Report

Chapter 2 studies the existing system and related literature, Chapter 3 lists the requirements, Chapter 4 presents the design, Chapter 5 the implementation (including integrations, the Custom API and the AI features), Chapter 6 testing, Chapter 7 deployment, security and governance, Chapter 8 results and Chapter 9 concludes.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

In the existing process the placement officer circulates each drive by e-mail; students reply through a Google Form; the responses are pasted into a master spreadsheet; eligibility is checked by sorting on CGPA; interview lists are sent by e-mail; and offers are tracked in a separate workbook that is updated at the end of the season.

2.2 Limitations of the Existing System

- No enforcement of eligibility, deadlines or duplicate applications at the time of entry.
- Status changes and interview schedules depend on manual e-mails.
- Several copies of the same data (form responses, master sheet, offer sheet) drift apart.
- No calendar entries for interviews; reminders are not sent.
- No quick, department-wise statistics and no machine-readable access for other systems.

2.3 Low-Code versus Traditional Development

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code)	Low-Code on Zoho Creator
Development time	About 8–10 weeks (estimate)	About 3 weeks (late Oct – 10 Nov 2026)
Skills	Front-end, back-end, SQL, OAuth, DevOps	Data modelling, Deluge, Zoho configuration
Hosting & security	Server, database, SSL and authentication built by hand	Hosted by Zoho; login, roles and OAuth built in
Integrations	Hand-written API clients and token storage	Connections + invokeurl / zoho.sheet tasks
AI	Separate ML stack and model serving	Zia keyword extraction and Zia prediction field
Mobile	Separate mobile app	Creator mobile app for forms and reports
Limits	Unlimited customisation	Bound to Creator's components and Deluge

2.4 Literature Survey

Table 2.2: Summary of Literature Survey

S.No.	Source & Year	Key Findings	Relevance to this Project
1	Bock & Frank, 2021 [3]	Defines low-code platforms as model-driven environments with configurable data, UI and logic layers.	Frames the forms–workflows–Deluge split used here.
2	Sahay et al., 2020 [7]	Feature taxonomy for comparing low-code platforms.	Checklist for platform feature coverage (Appendix A).
3	Di Ruscio et al., 2022 [8]	Low-code shares its foundations with model-driven engineering.	Model-first design before scripting (Chapter 4).
4	Hosmer, Lemeshow & Sturdivant, 2013 [12]	Logistic models predict a binary outcome from a few explanatory variables.	Basis of the synthetic training data for the Zia prediction model.
5	Zoho Corporation [5], [10], [11]	Documentation of Creator connections, Custom APIs, Zia and Deluge integration tasks.	Used for every integration and AI feature.

2.5 Emerging Trends in Low-Code Development

Low-code platforms now ship AI as a building block: pre-trained services such as keyword extraction and no-code model training such as Zia prediction fields, alongside API-first features that let an application publish its own endpoints. This project uses all three: a pre-trained NLP service, a trained prediction model and a Custom API.

2.6 Proposed System

The proposed system keeps every placement record in one Creator application. Deluge validations stop invalid applications at entry; workflows keep statuses consistent and notify students; interviews are pushed to Zoho Calendar and confirmed results to Zoho Sheet; a Custom API publishes summary statistics; and Zia assists the officer with a skill-match score and a predicted outcome.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Key Responsibilities in the Application
Placement Officer (Administrator)	Manages students, companies and opportunities; screens applications; schedules interviews; records results; runs suitability checks; views the dashboard.
Student	Maintains own profile, applies for open opportunities, receives e-mail updates (Student permission profile).
Recruiter / Company contact	External; supplies job descriptions and interview outcomes to the officer.
Head of Department / Management	Reads dashboard and Custom API statistics (read-only).

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Priority
FR-01	Register student profiles with unique roll number, department, year, CGPA, skills and résumé	High
FR-02	Maintain companies and their contact details	High
FR-03	Post opportunities with job type, description, eligibility, minimum CGPA, package, deadline and status	High
FR-04	Accept an application only for an Open opportunity before its deadline, from an eligible student, once	High
FR-05	E-mail the officer and student on a new application and on every status change	High
FR-06	Record interview rounds; mark the application Interview Scheduled and notify the student	High
FR-07	Create a Zoho Calendar event for every interview	Medium
FR-08	Record placement results and mark the application Selected	High
FR-09	Export each placement result to Zoho Sheet without duplicates	Medium
FR-10	Compute an explainable skill-match suitability score using Zia keyword extraction	Medium
FR-11	Predict the likely outcome (Selected / Not Selected) with a trained Zia prediction model	Medium
FR-12	Expose department-wise placement statistics through a secured Custom API	Medium
FR-13	Show KPI cards, list/Kanban reports and an applications-by-status chart	High
FR-14	Remind students one day before an interview	Low

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Security	OAuth connections with limited scopes; Custom API restricted to administrators via OAuth2
NFR-02	Usability	Clear validation messages; quick links from the dashboard to every form
NFR-03	Performance	Dashboard uses aggregate count() queries; one duplicate lookup before each Sheet insert
NFR-04	Maintainability	Integration and AI logic in named, reusable functions
NFR-05	Responsible AI	AI output is advisory, explained, and never changes an application status

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator, India data centre (creatorapp.zoho.in); Deluge scripting
Zoho services	Zoho Mail (sendmail), Zoho Calendar, Zoho Sheet, Zia AI (keyword extraction, prediction)
API testing	Deluge Execute console; Creator Custom API builder
Client	Chrome / Edge / Firefox; Zoho Creator mobile app; 8 GB RAM PC

3.5 Use Case Model

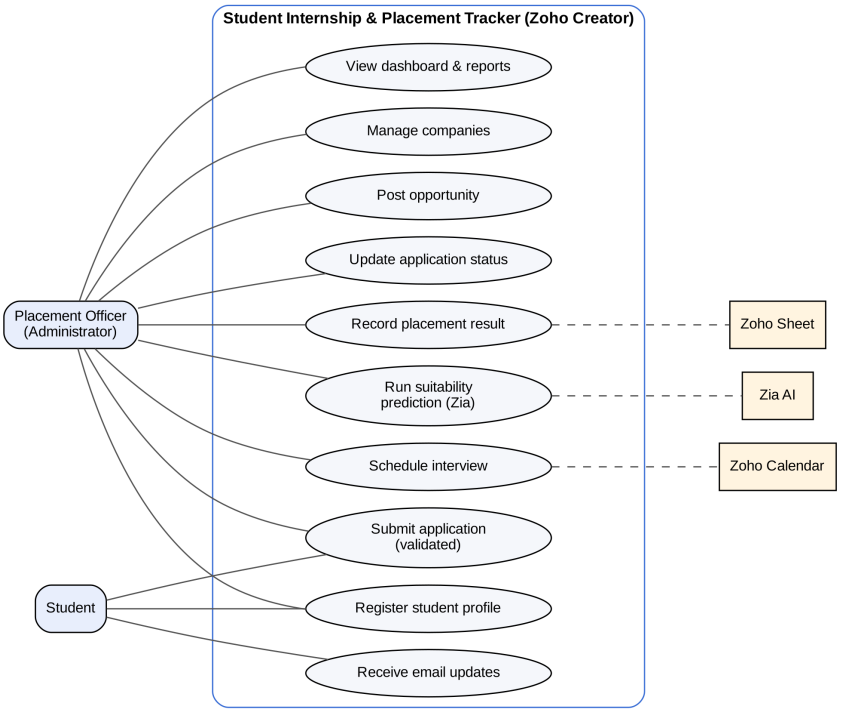


Figure 3.1: Use Case Diagram of the Application

Table 3.5: Use Case Description – Submit Application

Field	Description
Actor / Pre-condition	Student or placement officer; the student and an opportunity exist
Main flow	Open Applications form → choose student and opportunity → Submit → Validate_Application checks status, deadline, CGPA and duplicates → record saved as Applied → Notify_New_Application e-mails officer and student
Alternate flow	Closed / On Hold drive, past deadline, low CGPA or duplicate: alert shown and submit cancelled
Post-condition / FR	Application visible in reports and dashboard; FR-04, FR-05

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

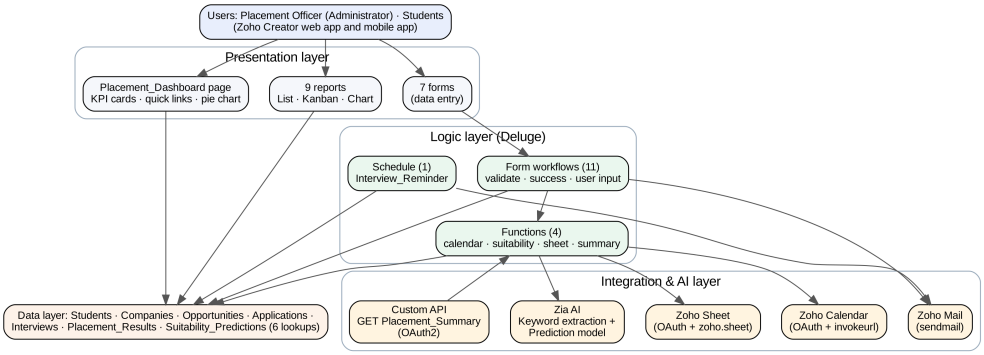


Figure 4.1: System Architecture of the Application

Users work through forms, reports and the dashboard page. Eleven form workflows and one schedule implement the rules; four Deluge functions hold the reusable integration and AI logic. The integration layer talks to Zoho Mail, Zoho Calendar, Zoho Sheet and Zia, and the Custom API exposes the summary function to authorised callers.

4.2 Data Model

The model has seven forms joined by six lookup fields. Applications is the central transaction entity linking a student to an opportunity; interviews and results hang off an application; suitability predictions link a student to an opportunity independently, so an assessment can be made before applying.

Table 4.1: Entities and Relationships

Entity (Form)	Related To	Relationship / Implemented Using
Students	Applications, Suitability_Predictions	One-to-many; Students lookup
Companies	Opportunities	One-to-many; Companies lookup
Opportunities	Applications, Suitability_Predictions	One-to-many; Opportunity / Opportunities lookup
Applications	Interviews, Placement_Results	One-to-many; Applications lookup
Interviews	Applications	Many-to-one
Placement_Results	Applications	Many-to-one (one result per selected application)
Suitability_Predictions	Students, Opportunities	Many-to-one on each; holds AI outputs

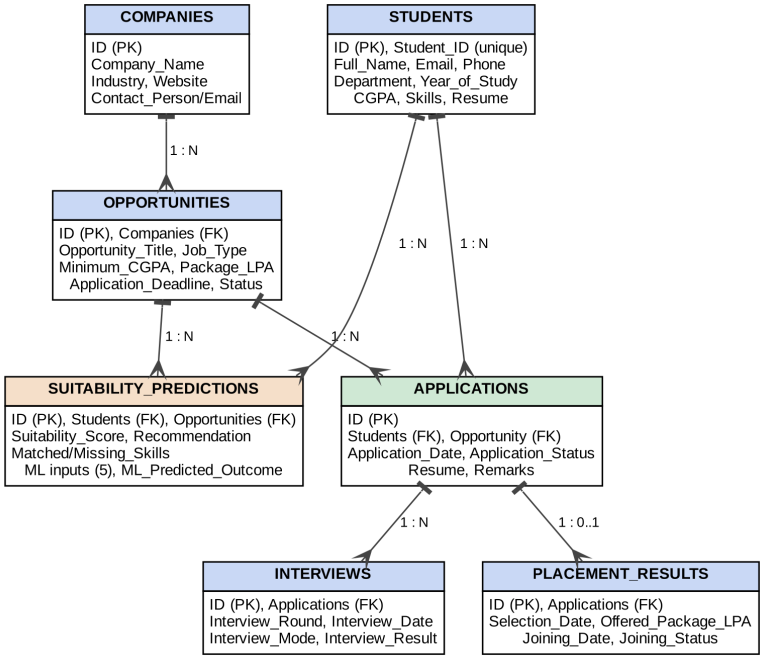


Figure 4.2: Entity–Relationship Diagram

4.3 Form Design

Tables 4.2 to 4.4 give the field specification of the main forms; Table 4.5 summarises all seven forms (the Suitability_Predictions fields are described in Section 5.9).

Table 4.2: Field Specification – Students Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default
Roll No.	Student_ID	Single Line	Yes	Unique
Full Name	Full_Name	Single Line	Yes	—
Email	Email	Email	Yes	Email format
Phone	Phone_Number	Phone	No	—
Department	Department	Drop Down	Yes	CSE, IT, ECE, EEE, Mechanical
Year of Study	Year_of_Study	Drop Down	Yes	First – Fourth Year
CGPA	CGPA	Decimal	Yes	Used by validation and AI
Skills	Skills	Multi Line	No	Comma-separated; used by AI
Resume	Resume	File Upload	No	—

Table 4.3: Field Specification – Opportunities Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default
Opportunity Title	Opportunity_Title	Single Line	Yes	—
Company	Companies	Lookup	Yes	Lookup to Companies
Job Type	Job_Type	Drop Down	Yes	Internship, Full-Time, Part-Time, Graduate Trainee
Job Description	Job_Description	Multi Line	No	Input to Zia keyword extraction
Eligibility Criteria	Eligibility_Criteria	Multi Line	No	Input to Zia keyword extraction
Minimum CGPA	Minimum_CGPA	Decimal	No	Checked by Validate_Application
Location	Location	Single Line	No	—
Package (LPA)	Package_LPA	Decimal	No	—
Application Deadline	Application_Deadline	Date-Time	No	Checked by Validate_Application
Status	Opportunity_Status	Drop Down	Yes	Open, Closed, On Hold

Table 4.4: Field Specification – Applications Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default
Student	Students	Lookup	Yes	Lookup to Students
Opportunity	Opportunity	Lookup	Yes	Lookup to Opportunities
Application Date	Application_Date	Date-Time	Yes	Default: current time
Application Status	Application_Status	Drop Down	Yes	Applied, Under Review, Shortlisted, Interview Scheduled, Selected, Rejected, Withdrawn
Resume	Resume	File Upload	No	—
Remarks	Remarks	Multi Line	No	Included in status e-mail

Table 4.5: Summary of All Forms

Form	Key Fields and Types	Fields
Students	See Table 4.2	9
Companies	Company_Name, Industry (Drop Down: IT, Banking and Finance, Manufacturing, Healthcare, Consulting, Education), Website (URL), Contact_Person, Contact_Email, Contact_Phone, Company_Location, Company_Description	8
Opportunities	See Table 4.3	10
Applications	See Table 4.4	6

Form	Key Fields and Types	Fields
Interviews	Applications (Lookup), Interview_Round (Online Assessment, Technical Round 1/2, HR Round, Final Interview), Interview_Date (Date), Interview_Mode (Online, In Person, Telephone), Interview_Result (Pending, Passed, Failed, Rescheduled), Interview_Remarks	6
Placement_Results	Applications (Lookup), Selection_Date, Offered_Package_LPA, Joining_Date, Joining_Status (Offer Received, Accepted, Declined, Joined, Not Joined), Remarks	6
Suitability_Predictions	Students / Opportunities lookups; Suitability_Score, Recommendation, Matched_Skills, Missing_Skills, AI_Extracted_Requirements, Explanation, Method; five ML inputs; ML_Predicted_Outcome (Zia Prediction)	15

4.4 Process and Workflow Design

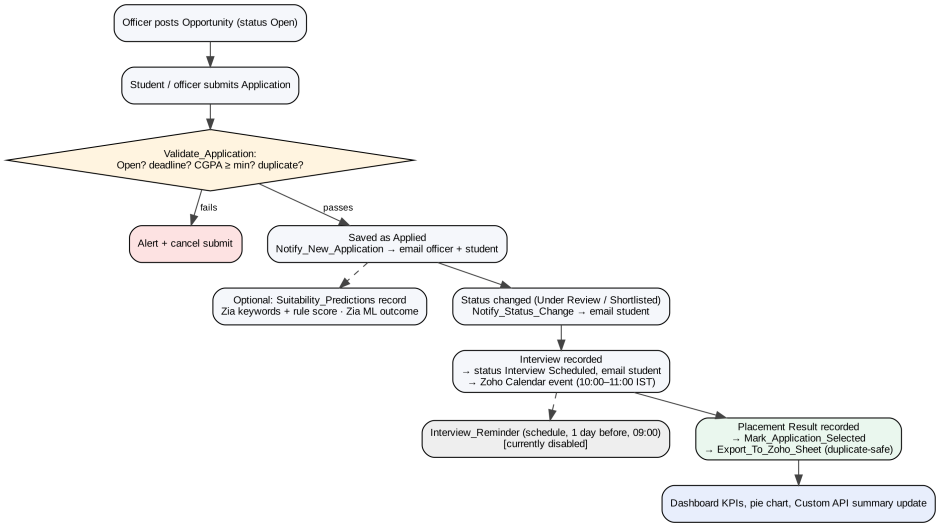


Figure 4.3: Workflow / Process Flow of an Application

Validation runs before save; notification, calendar, selection and export workflows run on success; the two user-input workflows fill the ML inputs while the suitability form is being completed; and one schedule is designed to send reminders a day before each interview.

4.5 User Interface and Navigation Design

The sidebar lists the dashboard first and then each form with its reports, in the order of the placement cycle. The dashboard gives one-click access to every form and report (Figure 4.4).

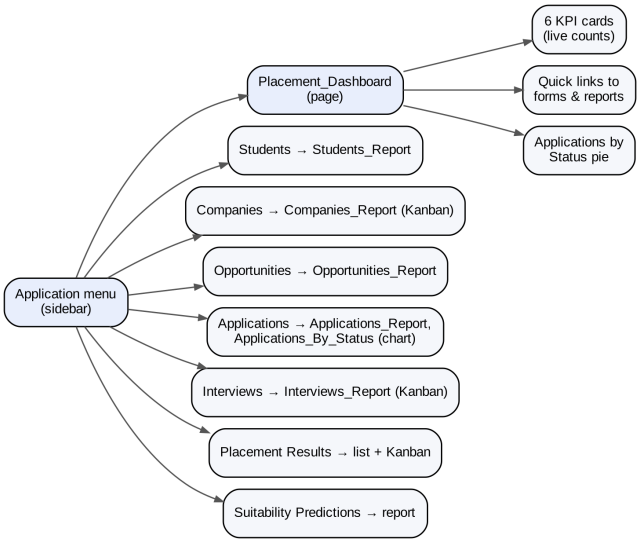


Figure 4.4: Navigation Structure of the Application

4.6 Access Control Design

Table 4.6: Role–Permission Matrix

Module	Placement Officer (Admin)	Student	Management (Read)
Students	Full	Add / edit own	View
Companies, Opportunities	Full	View	View
Applications	Full	Add / view own	View
Interviews, Placement_Results	Full	View own	View
Suitability_Predictions	Full	—	View
Dashboard, Custom API	Full / API access	—	Dashboard only

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The application “Student Internship & Placement Tracker app” (link name student-internship-placement-tracker-app) was built in the India data centre under the account sujit.s.2024.cse_rajalaks896. The schema was defined form by form and checked in the Schema Builder (Figure 5.1); scripts were maintained in the Application IDE.



Figure 5.1: Application Schema in the Schema Builder

5.2 Forms and Data Capture

The seven forms use thirteen field types: Single Line, Multi Line, Email, Phone, URL, Decimal, Number, Drop Down, Date, Date-Time, File Upload, Lookup and Zia Prediction. The Students form (Figure 5.2) captures the CGPA and skills that drive both the validation and the AI features.

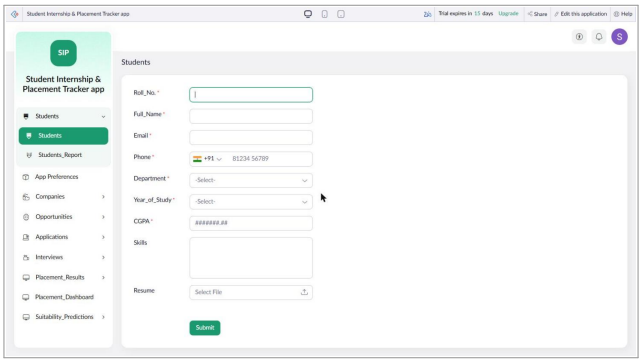


Figure 5.2: Students Form in Live Mode

5.3 Reports

Table 5.1: Reports Implemented

Report(s)	Type	Source	Grouping / Purpose
Students_Report, Opportunities_Report, Applications_Report, Placement_Results_Report, Suitability_Predictions_Report	List	Each form	Search, filter and sort records
Companies_Report	Kanban	Companies	Columns by Industry
Interviews_Report	Kanban	Interviews	Columns by Interview_Round
Placement_Results_Report1	Kanban	Placement_Results	Columns by Joining_Status
Applications_By_Status	Chart (pie)	Applications	Count by Application_Status

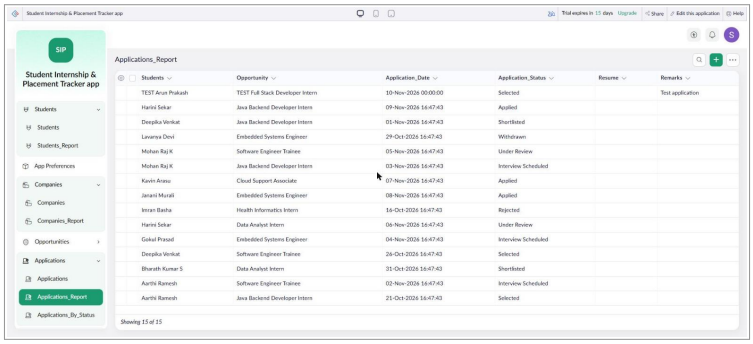


Figure 5.3: Applications_Report (List Report)

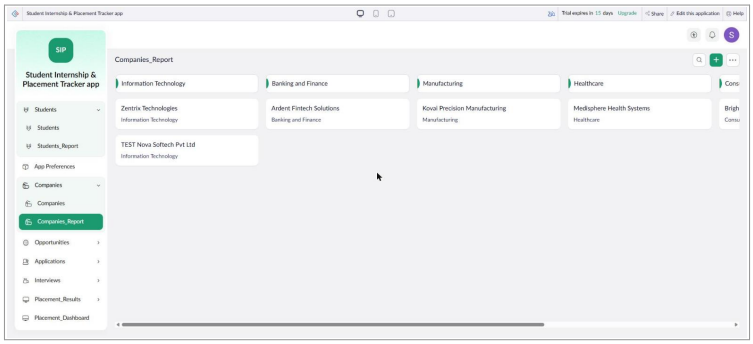


Figure 5.4: Companies_Report (Kanban by Industry)

5.4 Pages and Dashboards

Placement_Dashboard combines an HTML snippet and a chart component. Deluge count() queries feed six KPI cards (students, companies, opportunities, applications, interviews and placement results), a row of quick links opens every form and report, and the Applications_By_Status pie chart shows the funnel (Figure 5.5).

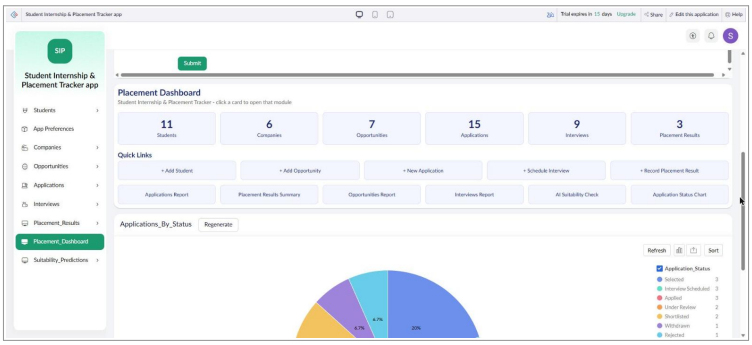


Figure 5.5: Placement_Dashboard: KPI Cards, Quick Links and Chart

5.5 Deluge Scripting

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Concepts Used
DS-01	Applications → On Validate	Open status, deadline, CGPA and duplicate checks	fetch, count(), alert, cancel submit
DS-02, DS-03	Applications → On Success / On Update of status	E-mail officer and student	sendmail, ifnull
DS-04	Interviews → On Success	Set Interview Scheduled; e-mail student	update via fetch, sendmail
DS-05	Interviews → On Success	Create Zoho Calendar event	function call, invokeurl
DS-06, DS-07	Placement_Results → On Success	Mark Selected; export to Zoho Sheet	zoho.sheet tasks
DS-08	Suitability_Predictions → On Success	Store AI score and explanation	Map, function call
DS-09, DS-10	Suitability_Predictions → On User Input	Fill the five ML input fields	input.<field>
DS-11	Schedule (1 day before Interview_Date)	Interview reminder e-mail	schedule, sendmail
DS-12 – DS-15	Functions	addInterviewToCalendar, computeSuitability, exportPlacementResult, getPlacementSummary	Map, List, for each, invokeurl, zoho.ai, return

Code 5.1: Application eligibility validation (DS-01)

```
opp = Opportunities[ID == input.Opportunity];
if(opp.count() > 0) {
  if(opp.Opportunity_Status != "Open") {
    alert "This opportunity is " + opp.Opportunity_Status + ". Applications are accepted only for Open
        opportunities.";
    cancel submit;
  } else if(opp.Application_Deadline != null && opp.Application_Deadline < zoho.currenttime) {
    alert "The application deadline for this opportunity has passed.";
    cancel submit;
  } else {
    stu = Students[ID == input.Students];
    if(stu.count() > 0 && opp.Minimum_CGPA != null && stu.CGPA < opp.Minimum_CGPA) {
      alert "Student CGPA (" + stu.CGPA + ") is below the minimum CGPA (" + opp.Minimum_CGPA + ") required
          for this opportunity.";
      cancel submit;
    } else {
```

```
      dup = Applications[Students == input.Students && Opportunity == input.Opportunity].count(ID);
      if(dup > 0) {
        alert "This student has already applied for this opportunity.";
        cancel submit;
      }
    }
  }
}
```

DS-01 stops an application in four cases, each with its own message: the opportunity is not Open, its deadline has passed, the student's CGPA is below the minimum, or the same student has already applied. All four were tested (Section 6.2).

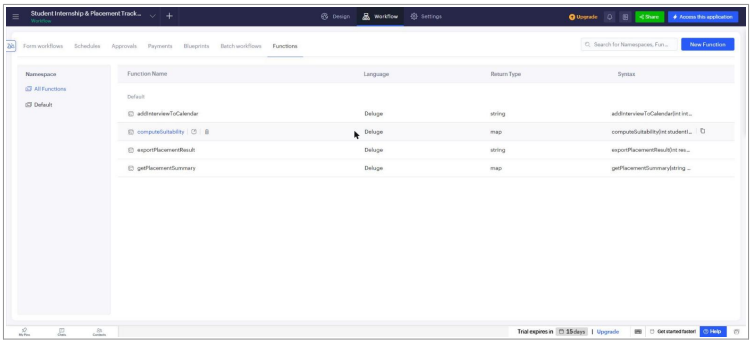


Figure 5.6: Deluge Functions of the Application

5.6 Workflows and Automation

Table 5.3: Workflows Implemented

Workflow	Type	Trigger	Action	Script
Validate_Application	Form	Applications, on validate	Block invalid applications	DS-01
Notify_New_Application	Form	Applications, on success	E-mail officer + student	DS-02
Notify_Status_Change	Form	Update of Application_Status	E-mail student (cc officer)	DS-03
Notify_Interview_Scheduled	Form	Interviews, on success	Status + e-mail	DS-04
Add_Interview_To_Zoho_Calendar	Form	Interviews, on success	Calendar event	DS-05
Mark_Application_Selected	Form	Placement_Results, on success	Status = Selected	DS-06
Export_To_Zoho_Sheet	Form	Placement_Results, on success	Row in Zoho Sheet	DS-07
Run_Suitability_Prediction	Form	Suitability record created / edited	AI score + explanation	DS-08
Fill_ML_Inputs_On_Student / _On_Opportunity	Form	User input of Students / Opportunities	Fill model inputs	DS-09, 10
S	Form	Applications, on load	Empty placeholder (no actions)	—

Workflow	Type	Trigger	Action	Script
Interview_Reminder	Schedule	1 day before Interview_Date, 09:00 IST	Reminder e-mail (currently disabled)	DS-11
Approval / Blueprint / Batch	—	Not used	Status flow is linear and handled by form workflows	—

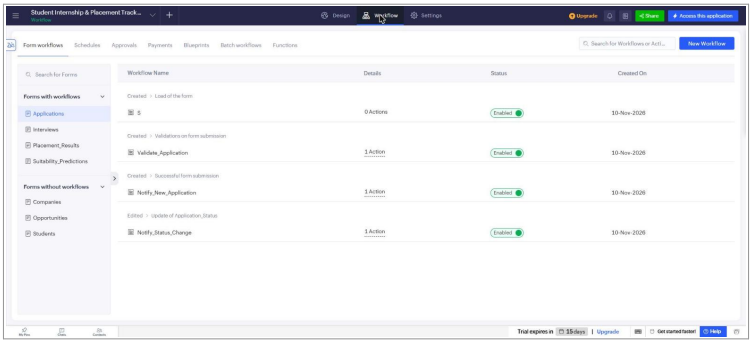


Figure 5.7: Form Workflows Configured on the Applications Form

5.7 Integrations and Custom API

Table 5.4: Integrations Implemented

Integration	Method	Authentication	Data Exchanged
Zoho Mail	sendmail (DS-02 – DS-04, DS-11)	Zoho account	Application, status and interview e-mails
Zoho Calendar [9]	invokeurl GET calendars, POST event (DS-05, DS-12)	OAuth connection zoho_calendar_connection	Event 10:00–11:00 IST with student, company, round, mode
Zoho Sheet	zoho.sheet.getRecords / createRecords (DS-07, DS-14)	OAuth connection zoho_sheet_connection	One row per result in Placement_Results_Export
Zia AI	zoho.ai.extractKeywords; Zia prediction field	Built into Creator	Opportunity text → keywords; 5 inputs → outcome
Custom API “Placement Summary”	GET /Placement_Summary (DS-15)	OAuth2, Admin only	Department-wise JSON statistics

5.7.1 Zoho Calendar Integration

When an interview is saved, Add_Interview_To_Zoho_Calendar calls addInterviewToCalendar. The function reads the user's calendars through the OAuth connection [1], picks the default calendar and posts an event. Because Interview_Date has no time component, a default 10:00–11:00 IST slot is used. The function returns CREATED, SKIPPED or FAILED so the result can be inspected in the Execute console.

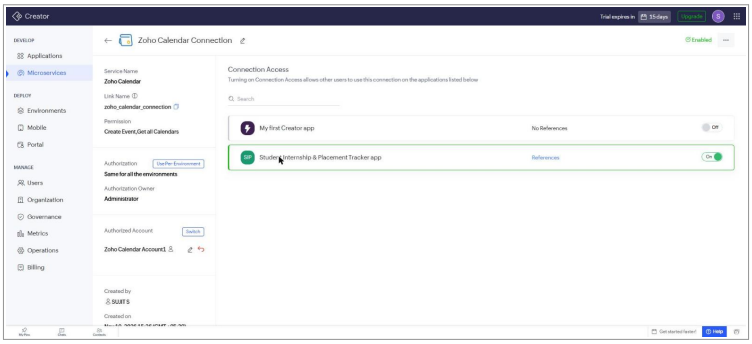


Figure 5.8: Connection Setup – Zoho Calendar (OAuth)

Code 5.2: Zoho Calendar event creation through a secured connection (DS-12, extract)

```
d = iv.Interview_Date.toString("yyyyMMdd");
dt = Map();
dt.put("timezone","Asia/Kolkata");
dt.put("start",d + "T100000+0530");
dt.put("end",d + "T110000+0530");
ev = Map();
ev.put("title","Interview: " + stu.Full_Name + " - " + opp.Opportunity_Title + " (" + iv.Interview_Round
    + ")");
ev.put("dateandtime",dt);
ev.put("location",ifnull(iv.Interview_Mode,""));
ev.put("description","Student: " + stu.Full_Name + " (" + stu.Student_ID + ")\nCompany: " + compName +
    "\nRound: " + iv.Interview_Round + "\nMode: " + iv.Interview_Mode);
params = Map();
params.put("eventdata",ev.toString());
resp = invokeurl [
  url :"https://calendar.zoho.in/api/v1/calendars/" + calUid + "/events"
  type :POST
  parameters:params
  connection:"zoho_calendar_connection"
];
if(resp.get("events") != null) {
  return "CREATED: calendar event for interview " + interviewId + " on " + iv.Interview_Date;
}
return "FAILED: " + resp;
```

5.7.2 Zoho Sheet Integration

Every placement result is copied to the sheet Placement_Results_Export (columns Record_ID, Student, Opportunity, Company, Selection_Date, Package_LPA, Joining_Status, Exported_On). The record key is stored as “PR-<id>” so the spreadsheet keeps the long Creator ID as text, and the function first searches the sheet for that key, so re-saving a record never adds a duplicate row.

Code 5.3: Duplicate-safe export to Zoho Sheet (DS-14, extract)

```
key = "PR-" + resultId;
q = Map();
q.put("criteria","\"Record_ID\"=\"" + key + "\"");
existing = zoho.sheet.getRecords(resId,sheetName,q,"zoho_sheet_connection");
if(existing.get("status") != "success") {
  return "FAILED (lookup): " + existing;
}
recs = existing.get("records");
if(recs != null && recs.size() > 0) {
  return "SKIPPED: " + key + " is already in the sheet - no duplicate row added";
```

5.7.3 Custom API – Placement Summary

A Custom API was created in Creator's Microservices section so that other systems (for example a department website or a management report) can read placement statistics without opening the application. Its configuration is summarised in Table 5.5; the API calls the Deluge function getPlacementSummary, which returns counts by application status, open opportunities, students placed and the average and highest package for one department, or for all departments when the parameter is empty.

Table 5.5: Custom API Configuration

Property	Value
Name / Link name	Placement Summary / Placement_Summary
Method	GET
Authentication / User scope	OAuth2 / Admin only
Request parameter	department (STRING, query parameter, optional)
Function	getPlacementSummary(string department) → map
Response	Standard: {"code": 3000, "result": {...}}
Application / Namespace	Student Internship & Placement Tracker app / Default
Status	Enabled (created 10 Nov 2026)

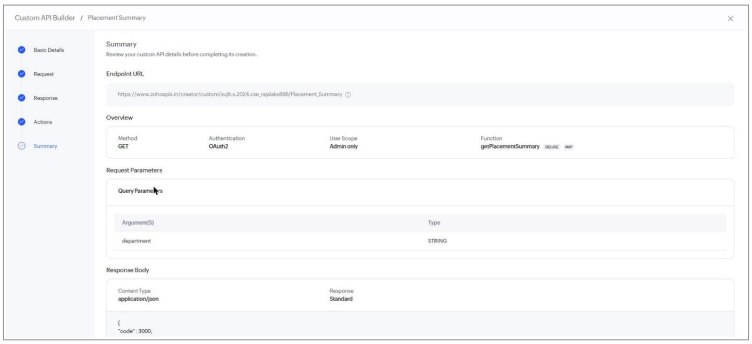


Figure 5.9: Custom API Builder – Placement Summary

Code 5.4: Example request and response of the Placement Summary API

```
GET https://www.zohoapis.in/creator/custom/sujit.s.2024.cse_rajalaks896/
    Placement_Summary?department=Computer Science and Engineering
Authorization: Zoho-oauthtoken <access token>

{
    "code": 3000,
    "result": {
        "department": "Computer Science and Engineering",
        "total_students": 5,
        "total_applications": 7,
        "applications_by_status": {
            "Applied": 2, "Under Review": 1, "Shortlisted": 1,
            "Interview Scheduled": 1, "Selected": 2,
            "Rejected": 0, "Withdrawn": 0
        },
        "open_opportunities": 5,
        "students_placed": 2,
        "average_package_lpa": 5.50,
        "highest_package_lpa": 6.50,
        "generated_on": "10-Nov-2026 16:51"
    }
}
```

The response shown is the actual output of getPlacementSummary for the Computer Science and Engineering department obtained in the Execute console (Figure 5.10), wrapped in Creator's standard response format.

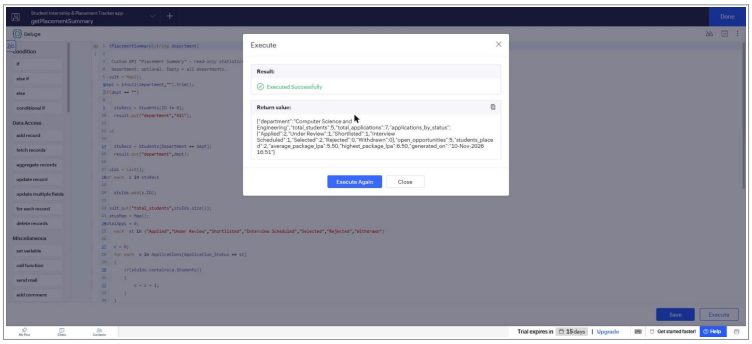


Figure 5.10: Execution Output of getPlacementSummary

5.8 User Management and Role-Based Access

The account has one user, SUJIT S (240701545), with the Super Admin role, who acts as the placement officer. The application has the permission profiles Read, Write and Student (Figure 5.11); the Student profile is intended for student users once they are invited. Because no additional users were added during development, the separation of roles in Table 4.6 is enforced by design rather than tested with separate accounts.

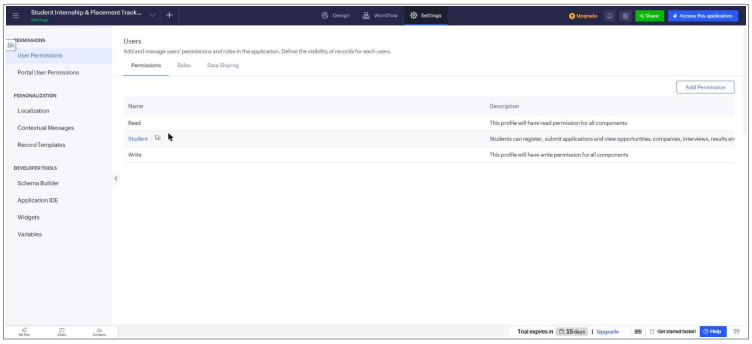


Figure 5.11: Permission Profiles: Read, Student and Write

5.9 AI and Analytics Features

The application uses Zia in two complementary ways. Both are **decision support only**: neither changes an application's status.

5.9.1 Zia Keyword Extraction and Rule-Based Suitability Score

computeSuitability sends the opportunity's Job_Description and Eligibility_Criteria (up to 4,900 characters) to *zoho.ai.extractKeywords*, a pre-trained Zia NLP service, which returns up to 15 ranked key phrases. Generic words ("skills", "intern", "CGPA" …) are filtered out and the remaining phrases are treated as requirements. Each requirement is matched against the student's comma-separated skills, by whole phrase or by any word. The score is:

Score = (matched requirements ÷ total requirements) × 70 + 30 if CGPA ≥ minimum CGPA

The recommendation is *Strong Match* for a score of 70 or more, *Review* for 40–69 and *Weak Match* below 40 or whenever the CGPA is below the cut-off; *Insufficient Data* is returned if skills, CGPA or extractable requirements are missing. A plain-language explanation and the method are stored with the result.

Code 5.5: Scoring and recommendation rule (DS-13, extract)

```
coverage = matched.size().toDecimal() / reqSkills.size();
skillPts = coverage * 70;
minC = opp.Minimum_CGPA;
cgpaOk = true;
if(minC != null && stu.CGPA < minC) {
  cgpaOk = false;
}
cgpaPts = 0;
if(cgpaOk) {
  cgpaPts = 30;
}
score = (skillPts + cgpaPts).round(1);
if(!cgpaOk) {
  rec = "Weak Match";
} else if(score >= 70) {
  rec = "Strong Match";
} else if(score >= 40) {
```

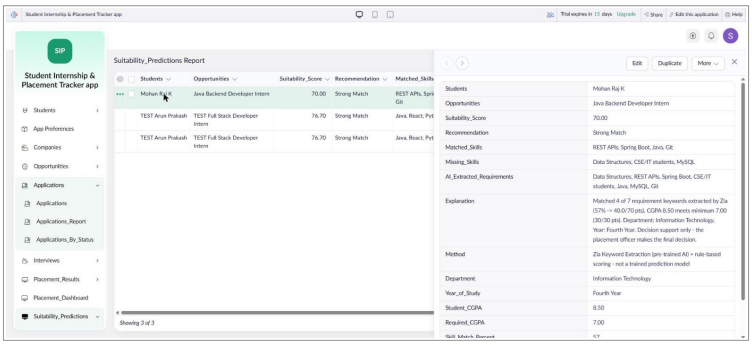


Figure 5.12: Suitability Result: 70.00, Strong Match, with Matched and Missing Skills

For student Mohan Raj K and the Java Backend Developer Intern opportunity, Zia extracted seven requirements; four (REST APIs, Spring Boot, Java, Git) matched, giving 4/7 × 70 = 40 points plus 30 points for CGPA, i.e. 70.00 and *Strong Match*. Missing skills were Data Structures and MySQL; Zia also returned "CSE/IT students", which is an eligibility phrase rather than a skill — a known limitation of generic keyword extraction (Section 8.4).

5.9.2 Zia Prediction Model “Suitability_Prediction”

A Zia prediction model was trained in Creator's AI Modeler [2] to predict whether an application is likely to end as *Selected* or *Not Selected*. It is attached to the field ML_Predicted_Outcome of the Suitability_Predictions form and uses five dependent fields that the Fill_ML_Inputs workflows fill automatically when a student and an opportunity are chosen.

Table 5.6: Zia Prediction Model Details

Property	Value
Model name / type	Suitability_Prediction / Prediction (binary classification)
Predicted field	ML_Predicted_Outcome (Selected / Not Selected)
Input (dependent) fields	Department, Year_of_Study, Student_CGPA, Required_CGPA, Skill_Match_Percent
Training data	CSV of 300 synthetic records: 191 Selected, 109 Not Selected
Data generation rule	Logistic rule: z = 2.2 × (CGPA − required CGPA) + 0.06 × (skill % − 50) + year term (−1.5, −0.7, +0.3, +0.8 for years 1–4) + 0.4 for CSE/IT; fixed random seed 42
Zia model score	96 (version 1, created 10 Nov 2026, published)
Deployed to	Suitability_Predictions.ML_Predicted_Outcome in this application

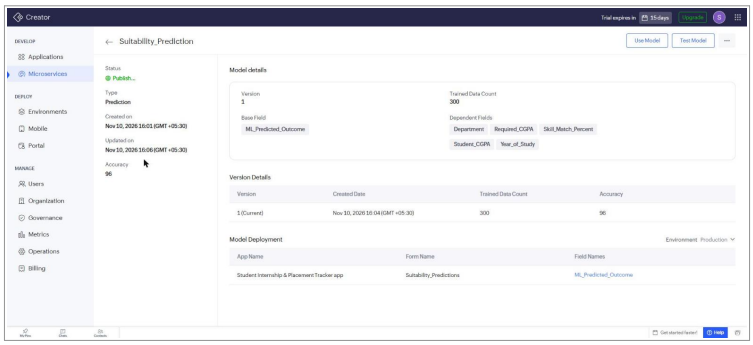


Figure 5.13: Zia Prediction Model Details in the AI Modeler

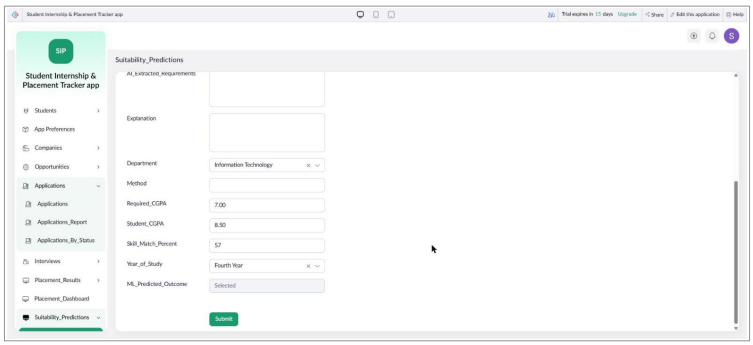


Figure 5.14: Suitability_Predictions Form with Auto-Filled Inputs and ML Outcome

Table 5.7: Spot Test of the Prediction Model (8 Cases Entered in the Live Form)

#	Dept / Year	CGPA	Req. CGPA	Skill %	Zia Prediction	Rule Expectation
1	CSE / Third	8.4	7.0	67	Selected	Selected
2	CSE / Third	6.0	7.0	5	Not Selected	Not Selected
3	IT / Fourth	8.5	7.0	57	Selected	Selected
4	IT / Fourth	6.2	7.5	20	Not Selected	Not Selected
5	IT / Fourth	7.0	7.0	40	Not Selected	Selected (borderline, z = 0.6)
6	IT / Fourth	9.0	8.0	10	Selected	Selected
7	IT / First	7.5	6.0	80	Selected	Selected
8	IT / Second	6.8	6.5	30	Not Selected	Not Selected

The model agreed with the generating rule in 7 of 8 cases; the one disagreement is a borderline case with exactly the required CGPA and a low skill match. Zia's score of 96 is its own training metric and the model has been trained only on synthetic data, so it is presented as a demonstration of the platform's ML capability, not as a validated placement predictor. Analytics are provided by the KPI cards, the pie chart and the Custom API statistics.

5.10 Pro-Code Extension

No external code was required: Deluge, connections, Zia and the Custom API covered every need. A pro-code extension would suit a résumé-parsing service, a public recruiter portal with custom UI, or training a model on real placement history outside Creator.

5.11 Mobile and Portal Access

All forms and the list and Kanban reports are available in the Zoho Creator mobile app with the same validations and workflows. No customer portal was configured; students would be invited as application users with the Student profile.

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Unit tests exercised each validation rule with valid and invalid inputs; integration tests checked each effect in the receiving system (inbox, Zoho Calendar, Zoho Sheet); functions were tested in the Deluge Execute console with info statements and returned status strings; the AI features were checked against hand-computed results; and the dashboard counts were compared with the reports. Role-based tests were not possible with a single user.

6.2 Test Cases

Table 6.1: Test Cases

TC	FR	Test / Input	Expected Result	Actual Result	Status
01	FR-01	Register student with valid data	Saved	Saved (11 students)	Pass
02	FR-04	Apply to a Closed opportunity	Refused with message	“This opportunity is Closed…”	Pass
03	FR-04	Student CGPA 6.80, minimum 7.00	Refused with message	CGPA message shown	Pass
04	FR-04	Same student, same opportunity again	Refused	“already applied” shown	Pass
05	FR-04, 05	Valid application	Saved as Applied; e-mails sent	As expected	Pass
06	FR-05	Change status to Shortlisted	E-mail to student	Sent	Pass
07	FR-06	Add interview for an application	Status Interview Scheduled	Updated	Pass
08	FR-07	Interview on 12-Nov-2026	Calendar event 10:00–11:00	CREATED; event visible	Pass
09	FR-08	Add placement result	Application Selected	Updated	Pass
10	FR-09	Placement result saved	Row in Zoho Sheet	EXPORTED: PR-<id>	Pass
11	FR-09	Export the same result again	Skipped, no duplicate	SKIPPED branch verified in code review	Reviewed
12	FR-10	Mohan Raj K vs Java Backend Intern	Score per rule	70.00, Strong Match	Pass
13	FR-11	8 spot cases (Table 5.7)	Plausible predictions	7 / 8 agree with rule	Pass
14	FR-12	getPlacementSummary("Computer Science and Engineering")	Correct JSON	5 students, 7 apps, 2 placed	Pass
15	FR-13	Dashboard vs reports	Counts match	Match	Pass

TC	FR	Test / Input	Expected Result	Actual Result	Status
16	FR-14	Interview reminder schedule	E-mail one day before	Schedule disabled	Not run
17	NFR-01	Student profile access	Restricted access	Not run (one user)	Not run

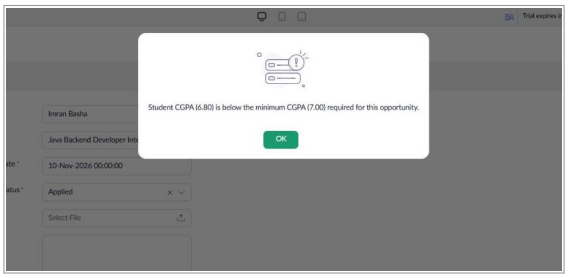


Figure 6.1: Validation Alert for CGPA below the Minimum (TC-03)

6.3 Debugging Deluge Scripts

Every function returns a status string (CREATED / SKIPPED / FAILED, EXPORTED / SKIPPED / FAILED) which the calling workflow writes with *info*, so failures can be read from the workflow log or reproduced in the Execute console (Figure 5.10). Integration responses are appended to FAILED messages to show the exact API error.

6.4 Defect Log

Table 6.2: Defect Log

Bug ID	Description / Root Cause	Fix Applied	Status
BUG-01	Application import failed: “Suitability_Prediction AI Model doesn't available” — the AI field referenced a model that did not exist	Removed AI properties, imported, trained the model, re-attached the field	Closed
BUG-02	IDE refused to change the field type to an AI field and refused field deletion	Deleted the field in the Form Builder, re-added it with AI properties, restored it in reports	Closed
BUG-03	Zia returns generic phrases (e.g. “CSE/IT students”) as requirements	Generic-word filter list; phrase still possible	Partly
BUG-04	Interviews and Placement_Results show the Applications lookup as a raw record ID	Set a display format (student + opportunity) on the lookup	Open
BUG-05	Interview_Reminder schedule is inactive	Enable the schedule in Workflows → Schedules	Open
BUG-06	Empty on-load workflow “S” on Applications	Delete the placeholder workflow	Open
BUG-07	Records inserted by script do not fire form workflows	Use form submissions for workflow-dependent data	Noted

6.5 User Acceptance Testing

Acceptance testing was carried out by the developer acting as placement officer, with realistic sample data (11 students, 6 companies, 7 opportunities, 15 applications, 9 interviews and 3 placement results). The feedback and the changes made are recorded in Table 6.3.

Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken
SUJIT S (developer)	Placement officer	Dashboard should show the whole funnel and give quick access to forms	KPI cards, quick links and pie chart added
SUJIT S (developer)	Placement officer	Interviews should appear in the calendar without retyping	Zoho Calendar integration added
SUJIT S (developer)	Placement officer	Offer details needed in a shareable sheet	Zoho Sheet export added
SUJIT S (developer)	Management	Statistics needed outside the app	Placement Summary Custom API added

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

The application runs in the Development environment and is accessed live at the URL in Appendix D. The Stage and Production environments of the account were not used (Figure 7.1); the recommended path for the next release is Development → Stage (acceptance test) → Production, with rollback by republishing the previous version.

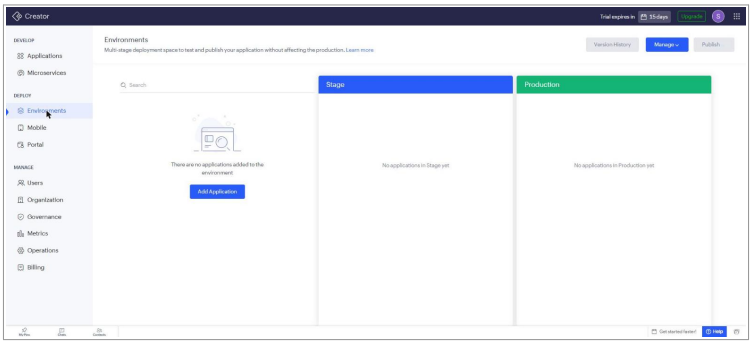


Figure 7.1: Environments – Development, Stage and Production

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Status
1	Test cases run (Table 6.1)	Partly (14 pass, 1 reviewed, 2 not run)
2	Connections authorised (Calendar, Sheet)	Done
3	Form workflows enabled	Done
4	Interview_Reminder schedule enabled	Pending
5	Zia model published and attached	Done
6	Custom API enabled and tested	Done
7	Users and profiles configured	Partly (one Super Admin)
8	Stage / Production publishing	Not used

7.2 Performance Optimisation

Table 7.2: Performance Optimisation Measures (timings not measured)

Optimisation Applied	Component	Effect
count() aggregates instead of loops	Dashboard KPI cards	Fewer records loaded
One criteria lookup before each insert	exportPlacementResult	No duplicate rows or repeated calls
Text truncated to 4,900 characters	computeSuitability	Stays within Zia request limits
Reusable functions called from workflows	All integrations	One code path to maintain

7.3 Security Features

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
OAuth connections; no keys in code	Zoho Calendar, Zoho Sheet	Credential leakage
OAuth2 + Admin-only user scope	Placement Summary Custom API	Unauthorised data access
Validation before save	Applications	Invalid or duplicate data
Advisory-only AI	Suitability workflows	Automated unfair decisions
Permission profiles	Read, Write, Student	Excess privileges for students

Field-level encryption, record-level criteria and IP restrictions are not yet configured.

7.4 Governance and Administration

System fields (Added Time, Added User, Modified Time) give an audit trail on every record; integration functions return explicit status strings; names follow fixed conventions (Notify_…, Fill_ML_Inputs_…, zoho_…_connection); the AI method is stored with every suitability result; and reports can be exported to Excel. Only the Super Admin can change the schema, workflows, connections and the Custom API.

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The live application holds 11 students, 6 companies, 7 opportunities, 15 applications, 9 interviews and 3 placement results. A complete scenario was run: (1) an application is validated and saved; (2) an interview is recorded and a Zoho Calendar event appears (Figure 8.1); (3) the placement result is saved, the application becomes Selected and a row is written to Zoho Sheet (Figure 8.2); (4) the dashboard and the Custom API reflect the new figures (Figure 8.3).

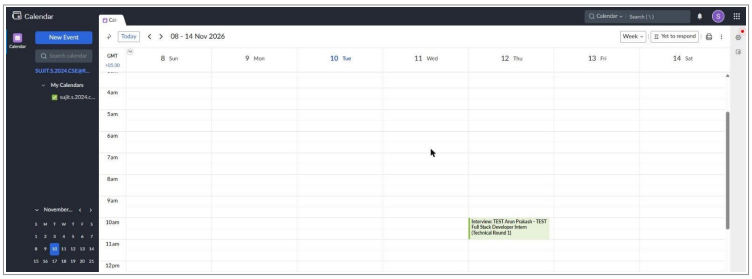


Figure 8.1: Scenario Step 2 – Interview Event Created in Zoho Calendar

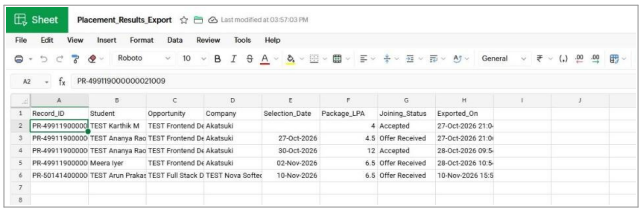


Figure 8.2: Scenario Step 3 – Placement Results Exported to Zoho Sheet

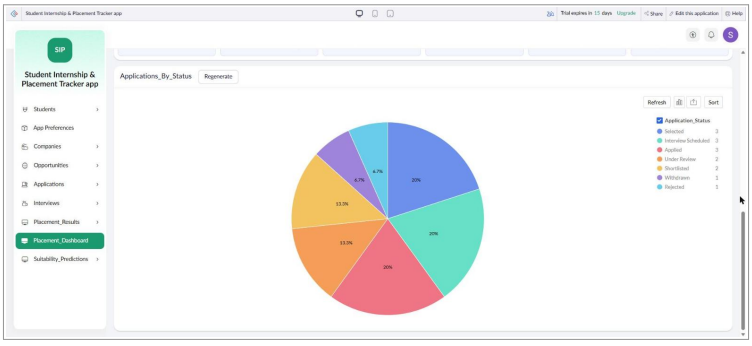


Figure 8.3: Scenario Step 4 – Applications by Status on the Dashboard

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective	Implemented Through	Evidence
1. Data model	4.2, 4.3, 5.2	7 forms, 6 lookups, 60 fields
2. Eligibility rules	5.5	TC-02 – TC-04
3. Automation	5.6	TC-05 – TC-09
4. Calendar & Sheet integrations	5.7	TC-08, TC-10
5. Custom API	5.7.3	TC-14
6. AI decision support	5.9	TC-12, TC-13
7. Reports & dashboard	5.3, 5.4	9 reports, 1 page; TC-15

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application
Eligibility check	Manual, at shortlisting	Automatic, before save
Student communication	Ad-hoc e-mails	Automatic e-mails on every event
Interview scheduling	E-mail lists	Zoho Calendar events
Offer tracking	Separate workbook	Placement_Results + Zoho Sheet export
Fit assessment	Reading every JD by hand	Zia keywords + score + ML prediction
Statistics	Counted by hand	Live dashboard and Custom API

8.4 Limitations of the Low-Code Approach

Generic keyword extraction cannot distinguish skills from eligibility phrases (BUG-03); the prediction model is trained on synthetic data and its 96 score is not a real-world accuracy; Interview_Date has no time, so calendar events use a fixed slot; script inserts do not fire workflows (BUG-07); lookups need display formats to be readable (BUG-04); and the application is tied to one vendor's platform and limits.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

No-code features covered forms, reports, the chart, connections, the AI Modeler, the Custom API definition and permissions; low-code Deluge covered eligibility rules, notifications, integrations, the suitability score and the API logic; pro-code would be justified only for résumé parsing, a custom recruiter portal or model training on real historical data.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The Student Internship & Placement Tracker replaces scattered spreadsheets with a single Zoho Creator application. Validation stops ineligible and duplicate applications at entry, workflows keep statuses and students up to date, interviews and offers flow to Zoho Calendar and Zoho Sheet through OAuth connections, a secured Custom API publishes placement statistics, and Zia adds explainable skill matching and a predicted outcome — always as advice to the placement officer, never as an automatic decision.

9.2 Future Enhancements

- Enable the interview reminder schedule and add an interview time field so calendar events use the real slot.
- Invite students with the Student profile and add record-level criteria so each student sees only their own records.
- Retrain the Zia model on real placement outcomes once enough history is collected, and report held-out accuracy.
- Add a recruiter portal, résumé parsing and Stage → Production deployment.

9.3 Learning Outcomes and Career Relevance

The project built citizen-developer skills (modelling the placement process), low-code developer skills (15 Deluge scripts, debugging platform errors), integration skills (OAuth connections, Zoho Sheet and Calendar APIs, publishing a Custom API) and responsible-AI judgement (keeping AI advisory and reporting it honestly) — directly relevant to Zoho Creator developer, automation engineer and implementation consultant roles.

REFERENCES

[1] Zoho Corporation, “Zoho Creator Help – Connections.” [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: 10 November 2026).

[2] Zoho Corporation, “Zia in Zoho Creator – AI Modeler and Prediction Field.” [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: 10 November 2026).

[3] A. C. Bock and U. Frank, “Low-Code Platform,” *Business & Information Systems Engineering*, vol. 63, no. 6, pp. 733–740, 2021, doi: 10.1007/s12599-021-00726-8.

[4] W. Rosam and O. Kalnichevski, *Low-Code Development: A Practical Guide*. New Delhi, India: BPB Publications.

[5] Zoho Corporation, “Zoho Creator Help Documentation.” [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: 10 November 2026).

[6] Zoho Corporation, “Deluge Script Reference Guide.” [Online]. Available: https://www.zoho.com/deluge/help/ (Accessed: 10 November 2026).

[7] A. Sahay, A. Indamutsa, D. Di Ruscio and A. Pierantonio, “Supporting the understanding and comparison of low-code development platforms,” in *Proc. 46th Euromicro SEAA*, 2020, pp. 171–178, doi: 10.1109/SEAA51224.2020.00036.

[8] D. Di Ruscio, D. Kolovos, J. de Lara, A. Pierantonio, M. Tisi and M. Wimmer, “Low-code development and model-driven engineering: Two sides of the same coin?,” *Software and Systems Modeling*, vol. 21, no. 2, pp. 437–446, 2022.

[9] Zoho Corporation, “Zoho Calendar API.” [Online]. Available: https://www.zoho.com/calendar/help/api/ (Accessed: 10 November 2026).

[10] Zoho Corporation, “Deluge – Zoho Sheet Integration Tasks.” [Online]. Available: https://www.zoho.com/deluge/help/ (Accessed: 10 November 2026).

[11] Zoho Corporation, “Custom APIs in Zoho Creator.” [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: 10 November 2026).

[12] D. W. Hosmer, S. Lemeshow and R. X. Sturdivant, *Applied Logistic Regression*, 3rd ed. Hoboken, NJ, USA: Wiley, 2013.

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature Coverage Checklist

Unit	Feature	Done	Where in the Application	Sec.	Fig./Code
I	Application & sections/menu	Y	Sidebar in placement order	5.1	Fig 4.4
II	Forms with ≥ 8 field types	Y	7 forms, 13 types	5.2	Tab 4.5
II	Lookup field	Y	6 lookups	4.2	Fig 4.2
II	Subform	N	Separate child forms	4.2	—
II	Validations	Y	Validate_Application	5.5	Code 5.1
II	List report: filter, sort	Y	5 list reports	5.3	Fig 5.3
II	Kanban / Calendar report	Partly	3 Kanban; no calendar report	5.3	Fig 5.4
II	Chart / pivot	Y	Applications_By_Status pie	5.3	Fig 8.3
II	Page / dashboard	Y	Placement_Dashboard	5.4	Fig 5.5
II	Deluge form events	Y	Validate, success, update, user input	5.5	Tab 5.2
II	Deluge functions, loops, collections	Y	4 functions	5.5	Fig 5.6
II	Email notifications	Y	4 sendmail workflows	5.6	App. C
II	Form workflow	Y	11 form workflows	5.6	Tab 5.3
II	Scheduled workflow	Partly	Interview_Reminder (disabled)	5.6	Tab 5.3
II	Approval / Blueprint / Batch	N	Not needed	5.6	—
II	Testing & debugging	Y	17 test cases, defect log	6	Tab 6.1
III	Integration with Zoho services	Y	Mail, Calendar, Sheet	5.7	Fig 8.1
III	Connection / invokeurl	Y	2 OAuth connections	5.7	Code 5.2
III	Custom API / REST	Y	Placement Summary (GET)	5.7.3	Fig 5.9
III	Users, roles, profiles	Partly	Read, Write, Student profiles	5.8	Fig 5.11
III	Record-level criteria / portal	N	Not configured	5.8	—
III	Zia / AI feature	Y	Keyword extraction + prediction model	5.9	Fig 5.13
III	Analytics	Y	Chart, KPIs, API statistics	5.4	Fig 5.5
IV	Performance optimisation	Partly	Applied, not measured	7.2	Tab 7.2
IV	Deployment across environments	N	Development only	7.1	Fig 7.1
IV	Security controls	Y	OAuth, API scope, validation	7.3	Tab 7.3
IV	Governance	Y	Audit fields, conventions	7.4	—
V	Pro-code extension	N	Not needed	5.10	—
V	Limitations & spectrum	Y	Real defects discussed	8.4–8.5	—
—	Mobile / portal access	Y	Creator mobile app (no portal)	5.11	—

Total features implemented: 21 / 30 (plus 4 partly).

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome (short)	Evidenced In	Self Level	Examiner Level
CO1	Principles and business value of low-code	Ch. 1, 2	3	
CO2	Develop apps with forms, reports, workflows, Deluge	Ch. 4, 5.1–5.6, 6	3	
CO3	Integrate services, APIs, role-based security	Ch. 4.6, 5.7–5.9	3	
CO4	Deploy, secure and govern across environments	Ch. 7	2	
CO5	Evaluate low-code, AI-assisted and pro-code paths	Ch. 5.9, 5.10, 8.4, 8.5, 9.3	3	

Levels: 1 – Low, 2 – Medium, 3 – High.

APPENDIX C – COMPLETE DELUGE SOURCE CODE

The scripts below are the complete Deluge source of the application as configured in the Application IDE, grouped by trigger. Comment lines have been removed.

Code C.1: Validate_Application – Applications, on validate (DS-01)

```
opp = Opportunities[ID == input.Opportunity];
if(opp.count() > 0) {
  if(opp.Opportunity_Status != "Open") {
    alert "This opportunity is " + opp.Opportunity_Status + ". Applications are accepted only for Open
        opportunities.";
    cancel submit;
  } else if(opp.Application_Deadline != null && opp.Application_Deadline < zoho.currenttime) {
    alert "The application deadline for this opportunity has passed.";
    cancel submit;
  } else {
    stu = Students[ID == input.Students];
    if(stu.count() > 0 && opp.Minimum_CGPA != null && stu.CGPA < opp.Minimum_CGPA) {
      alert "Student CGPA (" + stu.CGPA + ") is below the minimum CGPA (" + opp.Minimum_CGPA + ") required
          for this opportunity.";
      cancel submit;
    } else {
      dup = Applications[Students == input.Students && Opportunity == input.Opportunity].count(ID);
      if(dup > 0) {
        alert "This student has already applied for this opportunity.";
        cancel submit;
      }
    }
  }
}
```

Code C.2: Notify_New_Application – Applications, on success (DS-02)

```
stu = Students[ID == input.Students];
opp = Opportunities[ID == input.Opportunity];
studentName = "";
studentEmail = "";
if(stu.count() > 0) {
  studentName = stu.Full_Name;
  studentEmail = stu.Email;
}
oppTitle = "";
if(opp.count() > 0) {
  oppTitle = opp.Opportunity_Title;
}
sendmail [
  from :zoho.adminuserid
  to :zoho.adminuserid
  subject :"New Application: " + studentName + " - " + oppTitle
  message :"<p>A new application has been submitted.</p><p><b>Student:</b> " + studentName +
      "<br><b>Opportunity:</b> " + oppTitle + "<br><b>Date:</b> " + input.Application_Date +
      "<br><b>Status:</b> " + input.Application_Status + "</p>"
]
if(studentEmail != null && studentEmail != "") {
  sendmail [
    from :zoho.adminuserid
    to :studentEmail
    subject :"Application received - " + oppTitle
    message :"<p>Dear " + studentName + ",</p><p>Your application for <b>" + oppTitle + "</b> has been
        received. Current status: <b>" + input.Application_Status + "</b>.</p><p>Placement Cell</p>"
  ]
}
```

Code C.3: Notify_Status_Change – on update of Application_Status (DS-03)

```
stu = Students[ID == input.Students];
opp = Opportunities[ID == input.Opportunity];
if(stu.count() > 0 && stu.Email != null && stu.Email != "") {
  oppTitle = "";
  if(opp.count() > 0) {
    oppTitle = opp.Opportunity_Title;
  }
  sendmail [
    from :zoho.adminuserid
    to :stu.Email
    cc:zoho.adminuserid
    subject :"Application status updated: " + input.Application_Status
    message :"<p>Dear " + stu.Full_Name + ",</p><p>The status of your application for <b>" + oppTitle +
```

```
        "</b> has been updated to <b>" + input.Application_Status + "</b>.</p><p>Remarks: " +
        ifnull(input.Remarks,"-") + "</p><p>Placement Cell</p>"
  ]
}
```

Code C.4: Notify_Interview_Scheduled – Interviews, on success (DS-04)

```
app = Applications[ID == input.Applications];
if(app.count() > 0) {
  if(app.Application_Status != "Selected" && app.Application_Status != "Rejected") {
    app.Application_Status = "Interview Scheduled";
  }
  stu = Students[ID == app.Students];
  opp = Opportunities[ID == app.Opportunity];
  oppTitle = "";
  if(opp.count() > 0) {
    oppTitle = opp.Opportunity_Title;
  }
  if(stu.count() > 0 && stu.Email != null && stu.Email != "") {
    sendmail [
      from :zoho.adminuserid
      to :stu.Email
      cc:zoho.adminuserid
      subject :"Interview scheduled - " + oppTitle
      message :"<p>Dear " + stu.Full_Name + ",</p><p>Your <b>" + input.Interview_Round + "</b> interview
          for <b>" + oppTitle + "</b> is scheduled on <b>" + input.Interview_Date + "</b> (Mode: " +
          input.Interview_Mode + ").</p><p>All the best!<br>Placement Cell</p>"
    ]
  }
}
```

Code C.5: Add_Interview_To_Zoho_Calendar – Interviews, on success (DS-05)

```
res = thisapp.addInterviewToCalendar(input.ID);
info res;
```

Code C.6: Mark_Application_Selected – Placement_Results, on success (DS-06)

```
app = Applications[ID == input.Applications];
if(app.count() > 0) {
  app.Application_Status = "Selected";
}
```

Code C.7: Export_To_Zoho_Sheet – Placement_Results, on success (DS-07)

```
result = thisapp.exportPlacementResult(input.ID,"Sheet1");
info result;
```

Code C.8: Run_Suitability_Prediction – Suitability_Predictions, on success (DS-08)

```
r = thisapp.computeSuitability(input.Students,input.Opportunities);
rec = Suitability_Predictions[ID == input.ID];
if(rec.count() > 0) {
  rec.Suitability_Score = r.get("score");
  rec.Recommendation = r.get("recommendation");
  rec.Matched_Skills = ifnull(r.get("matched"),"");
  rec.Missing_Skills = ifnull(r.get("missing"),"");
  rec.AI_Extracted_Requirements = ifnull(r.get("ai_keywords"),"");
  rec.Explanation = r.get("explanation");
  rec.Method = r.get("method");
}
```

Code C.9: Fill_ML_Inputs_On_Student / _On_Opportunity – on user input (DS-09, DS-10)

```
if(input.Students != null && input.Opportunities != null) {
  stu = Students[ID == input.Students];
  opp = Opportunities[ID == input.Opportunities];
  input.Student_CGPA = stu.CGPA;
  input.Required_CGPA = ifnull(opp.Minimum_CGPA,0);
  input.Department = stu.Department;
  input.Year_of_Study = stu.Year_of_Study;
  r = thisapp.computeSuitability(input.Students,input.Opportunities);
  if(r.get("skill_pct") != null) {
    input.Skill_Match_Percent = r.get("skill_pct").toLong();
  }
}
```

Code C.10: Interview_Reminder – schedule, 1 day before Interview_Date at 09:00 IST (DS-11)

```
if(input.Interview_Result != "Passed" && input.Interview_Result != "Failed") {
  app = Applications[ID == input.Applications];
  if(app.count() > 0) {
    stu = Students[ID == app.Students];
    opp = Opportunities[ID == app.Opportunity];
    oppTitle = "";
    if(opp.count() > 0) {
      oppTitle = opp.Opportunity_Title;
    }
    if(stu.count() > 0 && stu.Email != null && stu.Email != "") {
      sendmail [
        from :zoho.adminuserid
        to :stu.Email
        cc:zoho.adminuserid
        subject :"Reminder: Interview tomorrow - " + oppTitle
        message :"<p>Dear " + stu.Full_Name + ",</p><p>This is a reminder that your <b>" +
            input.Interview_Round + "</b> interview for <b>" + oppTitle + "</b> is on <b>" +
            input.Interview_Date + "</b> (Mode: " + input.Interview_Mode + ").</p><p>Placement Cell</p>"
      ]
    }
  }
}
```

Code C.11: Function addInterviewToCalendar (DS-12)

```
string addInterviewToCalendar(int interviewId) {
  if(interviewId == 0) {
    interviewId = Interviews[ID != 0].maximum(ID);
  }
  iv = Interviews[ID == interviewId];
  if(iv.count() == 0 || iv.Interview_Date == null) {
    return "SKIPPED: interview or date missing";
  }
  app = Applications[ID == iv.Applications];
  stu = Students[ID == app.Students];
  opp = Opportunities[ID == app.Opportunity];
  compName = "";
  if(opp.Companies != null) {
    compName = Companies[ID == opp.Companies].Company_Name;
  }
  cals = invokeurl [
    url :"https://calendar.zoho.in/api/v1/calendars"
    type :GET
    connection:"zoho_calendar_connection"
  ];
  calUid = "";
  for each c in cals.get("calendars") {
    if(calUid == "" || c.get("isdefault") == true) {
      calUid = c.get("uid");
    }
  }
  if(calUid == "") {
    return "FAILED: no calendar found " + cals;
  }
  d = iv.Interview_Date.toString("yyyyMMdd");
  dt = Map();
  dt.put("timezone","Asia/Kolkata");
  dt.put("start",d + "T100000+0530");
  dt.put("end",d + "T110000+0530");
  ev = Map();
  ev.put("title","Interview: " + stu.Full_Name + " - " + opp.Opportunity_Title + " (" + iv.Interview_Round
      + ")");
  ev.put("dateandtime",dt);
  ev.put("location",ifnull(iv.Interview_Mode,""));
  ev.put("description","Student: " + stu.Full_Name + " (" + stu.Student_ID + ")\nCompany: " + compName +
      "\nRound: " + iv.Interview_Round + "\nMode: " + iv.Interview_Mode);
  params = Map();
  params.put("eventdata",ev.toString());
  resp = invokeurl [
    url :"https://calendar.zoho.in/api/v1/calendars/" + calUid + "/events"
    type :POST
    parameters:params
    connection:"zoho_calendar_connection"
  ];
  if(resp.get("events") != null) {
    return "CREATED: calendar event for interview " + interviewId + " on " + iv.Interview_Date;
  }
  return "FAILED: " + resp;
}
```

Code C.12: Function computeSuitability (DS-13)

```
map computeSuitability(int studentId, int oppId) {
  result = Map();
  result.put("method","Zia Keyword Extraction (pre-trained AI) + rule-based scoring - not a trained
      prediction model");
  stu = Students[ID == studentId];
  opp = Opportunities[ID == oppId];
  if(stu.count() == 0 || opp.count() == 0) {
    result.put("recommendation","Insufficient Data");
    result.put("explanation","Student or opportunity record not found.");
    return result;
  }
  missingInfo = List();
  skillsText = ifnull(stu.Skills,"").trim();
  if(skillsText == "") {
    missingInfo.add("student skills");
  }
  if(stu.CGPA == null) {
    missingInfo.add("student CGPA");
  }
  reqText = (ifnull(opp.Job_Description,"") + ". " + ifnull(opp.Eligibility_Criteria,"")).trim();
  if(reqText.length() > 4900) {
    reqText = reqText.subString(0,4900);
  }
  reqSkills = List();
  reqLower = List();
  generic = {"skill","skills","knowledge","intern","internship","interns","experience","candidate","candidat
      es","student","students","team","ability","plus","good","strong","work","role","job","company","hiring
      ","required","requirement","requirements","understanding","basic","developer","web
      developer","engineer","fresher","freshers","year","years","degree","eligibility","criteria","test","te
      st record","record","flow","opportunity","familiarity","exposure","preferred","must","minimum","cgpa"}
      ;
  if(reqText.length() > 3) {
    resp = zoho.ai.extractKeywords(reqText,15,"RANKED_KEYWORDS",2);
    if(resp.get("status") == 200) {
      items = resp.get("data").get("items");
      for each item in items {
        for each kw in item.get("keywords") {
          k = kw.get("keyword").trim();
          kl = k.toLowerCase();
          if(!generic.contains(kl) && !reqLower.contains(kl) && kl.length() > 1) {
            reqSkills.add(k);
            reqLower.add(kl);
          }
        }
      }
    }
  }
  result.put("ai_keywords",reqSkills.toString(", "));
  if(reqSkills.size() == 0) {
    missingInfo.add("opportunity skill requirements (none extracted from description/eligibility text)");
  }
  if(missingInfo.size() > 0) {
    result.put("recommendation","Insufficient Data");
    result.put("explanation","Cannot assess - missing: " + missingInfo.toString(", ") + ". Please complete
        the record and re-run. The placement officer makes the final decision.");
    return result;
  }
  stuWords = List();
  for each item in skillsText.toList(",") {
    it = item.trim().toLowerCase();
    if(it != "") {
      stuWords.add(it);
      for each w in it.toList(" ") {
        if(w.trim() != "") {
          stuWords.add(w.trim());
        }
      }
    }
  }
  matched = List();
  missing = List();
  for each s in reqSkills {
    sl = s.toLowerCase();
    isMatch = stuWords.contains(sl);
    if(!isMatch) {
      for each t in sl.toList(" ") {
        if(stuWords.contains(t.trim())) {
          isMatch = true;
        }
      }
    }
    if(isMatch) {
      matched.add(s);
    } else {
      missing.add(s);
    }
```

```
  }
  coverage = matched.size().toDecimal() / reqSkills.size();
  skillPts = coverage * 70;
  minC = opp.Minimum_CGPA;
  cgpaOk = true;
  if(minC != null && stu.CGPA < minC) {
    cgpaOk = false;
  }
  cgpaPts = 0;
  if(cgpaOk) {
    cgpaPts = 30;
  }
  score = (skillPts + cgpaPts).round(1);
  if(!cgpaOk) {
    rec = "Weak Match";
  } else if(score >= 70) {
    rec = "Strong Match";
  } else if(score >= 40) {
    rec = "Review";
  } else {
    rec = "Weak Match";
  }
  expl = "Matched " + matched.size() + " of " + reqSkills.size() + " requirement keywords extracted by Zia
      (" + (coverage * 100).round(0) + "% -> " + skillPts.round(1) + "/70 pts). ";
  if(minC == null) {
    expl = expl + "No minimum CGPA set (30/30 pts). ";
  } else if(cgpaOk) {
    expl = expl + "CGPA " + stu.CGPA + " meets minimum " + minC + " (30/30 pts). ";
  } else {
    expl = expl + "CGPA " + stu.CGPA + " is BELOW the minimum " + minC + " (0/30 pts) - flagged as Weak
        Match. ";
  }
  expl = expl + "Department: " + ifnull(stu.Department,"-") + ", Year: " + ifnull(stu.Year_of_Study,"-") +
      ". Decision support only - the placement officer makes the final decision.";
  result.put("score",score);
  result.put("skill_pct",(coverage * 100).round(0));
  result.put("recommendation",rec);
  result.put("matched",matched.toString(", "));
  result.put("missing",missing.toString(", "));
  result.put("explanation",expl);
  return result;
}
```

Code C.13: Function exportPlacementResult (DS-14)

```
string exportPlacementResult(int resultId, string sheetName) {
  resId = "gc4d1e4f633a95038440592dcab30f5e4e9c1";
  pr = Placement_Results[ID == resultId];
  if(pr.count() == 0) {
    return "FAILED: placement result " + resultId + " not found";
  }
  key = "PR-" + resultId;
  q = Map();
  q.put("criteria","\"Record_ID\"=\"" + key + "\"");
  existing = zoho.sheet.getRecords(resId,sheetName,q,"zoho_sheet_connection");
  if(existing.get("status") != "success") {
    return "FAILED (lookup): " + existing;
  }
  recs = existing.get("records");
  if(recs != null && recs.size() > 0) {
    return "SKIPPED: " + key + " is already in the sheet - no duplicate row added";
  }
  studentName = "";
  oppTitle = "";
  companyName = "";
  app = Applications[ID == pr.Applications];
  if(app.count() > 0) {
    stu = Students[ID == app.Students];
    if(stu.count() > 0) {
      studentName = stu.Full_Name;
    }
    opp = Opportunities[ID == app.Opportunity];
    if(opp.count() > 0) {
      oppTitle = opp.Opportunity_Title;
      comp = Companies[ID == opp.Companies];
      if(comp.count() > 0) {
        companyName = comp.Company_Name;
      }
    }
  }
  selDate = "";
  if(pr.Selection_Date != null) {
    selDate = pr.Selection_Date.toString("dd-MMM-yyyy");
  }
  row = Map();
  row.put("Record_ID",key);
```

```
  row.put("Student",studentName);
  row.put("Opportunity",oppTitle);
  row.put("Company",companyName);
  row.put("Selection_Date",selDate);
  row.put("Package_LPA",ifnull(pr.Offered_Package_LPA,""));
  row.put("Joining_Status",ifnull(pr.Joining_Status,""));
  row.put("Exported_On",zoho.currenttime.toString("dd-MMM-yyyy HH:mm"));
  resp = zoho.sheet.createRecords(resId,sheetName,row,Map(),"zoho_sheet_connection");
  if(resp.get("status") == "success") {
    return "EXPORTED: " + key + " (" + studentName + " - " + oppTitle + ")";
  }
  return "FAILED (insert): " + resp;
}
```

Code C.14: Function getPlacementSummary – used by the Custom API (DS-15)

```
map getPlacementSummary(string department) {
  result = Map();
  dept = ifnull(department,"").trim();
  if(dept == "") {
    stuRecs = Students[ID != 0];
    result.put("department","All");
  } else {
    stuRecs = Students[Department == dept];
    result.put("department",dept);
  }
  stuIds = List();
  for each s in stuRecs {
    stuIds.add(s.ID);
  }
  result.put("total_students",stuIds.size());
  statusMap = Map();
  totalApps = 0;
  for each st in {"Applied","Under Review","Shortlisted","Interview
      Scheduled","Selected","Rejected","Withdrawn"} {
    c = 0;
    for each a in Applications[Application_Status == st] {
      if(stuIds.contains(a.Students)) {
        c = c + 1;
      }
    }
    statusMap.put(st,c);
    totalApps = totalApps + c;
  }
  result.put("total_applications",totalApps);
  result.put("applications_by_status",statusMap);
  result.put("open_opportunities",Opportunities[Opportunity_Status == "Open"].count());
  placed = 0;
  pkgTotal = 0.0;
  maxPkg = 0.0;
  for each pr in Placement_Results[ID != 0] {
    app = Applications[ID == pr.Applications];
    if(app.count() > 0 && stuIds.contains(app.Students)) {
      placed = placed + 1;
      p = ifnull(pr.Offered_Package_LPA,0).toDecimal();
      pkgTotal = pkgTotal + p;
      if(p > maxPkg) {
        maxPkg = p;
      }
    }
  }
  avgPkg = 0.0;
  if(placed > 0) {
    avgPkg = (pkgTotal / placed).round(2);
  }
  result.put("students_placed",placed);
  result.put("average_package_lpa",avgPkg);
  result.put("highest_package_lpa",maxPkg);
  result.put("generated_on",zoho.currenttime.toString("dd-MMM-yyyy HH:mm"));
  return result;
}
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

Open https://creatorapp.zoho.in/sujit.s.2024.cse_rajalaks896/student-internship-placement-tracker-app and sign in with a Zoho account, or open the application in the Zoho Creator mobile app. The Placement_Dashboard opens first.

D.2 Instructions for the Placement Officer

- **Masters:** add companies (Companies), then opportunities with description, eligibility, minimum CGPA, deadline and status Open.
- **Applications:** add or review applications; change Application_Status as the drive progresses — the student is e-mailed automatically.
- **Interviews:** add an interview for an application; the status becomes Interview Scheduled and an event is added to Zoho Calendar.
- **Results:** add a Placement Result; the application becomes Selected and a row is added to the Placement_Results_Export sheet.
- **Suitability:** in Suitability_Predictions choose a student and an opportunity; the ML outcome appears immediately and the score, matched/missing skills and explanation are filled on save.
- **Statistics:** use the dashboard, or call GET Placement_Summary?department=<name> with an OAuth token.

D.3 Instructions for Students

- Complete the Students form with roll number, department, year, CGPA and a comma-separated list of skills; upload a résumé.
- Apply only to opportunities marked Open before the deadline; the form explains any rule that blocks the application.
- Watch your e-mail for status changes and interview schedules.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	SUJIT S	**Register No.**	240701545
Project Title	Student Internship & Placement Tracker	**Date**	_____

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		100	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner

Name: _____

Signature of External Examiner

Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	SUJIT S	**Register No.**	240701545
Department / Section	B.E. Computer Science and Engineering	**Date of Viva**	_____
Application URL Verified	Yes / No	**Demo Duration**	_____ min

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	100	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER

Name: _____

EXTERNAL EXAMINER

Name: _____